

CHAPTER 3: Research Methodology

3.1. Introduction

In this chapter, the system development methodology chosen for the BookMyBus Zambia Management System is described and the procedures used for information gathering and requirement analysis are outlined. To move from the current manual, paper-based ticketing system to a centralized digital platform, a structured engineering approach is required. This chapter justifies the use of the Agile Scrum methodology and explains the multi-staged data collection process, which begins with a technical analysis of existing systems and concludes with field research involving stakeholders at The Copperbelt University. The chapter further establishes the principles underlying the system's data structures and the interaction between its modules.

This chapter is structured as follows: Section 3.2 describes and justifies the chosen Agile Scrum methodology and the OOAD approach. Section 3.3 details the information gathering and analysis techniques used to elicit requirements. Section 3.4 presents the formal requirements specification, while Section 3.5 provides the system analysis using UML models. The chapter concludes with a summary of the findings.

3.2. Methodology

The project follows the Agile Scrum SDLC (Software Development Life Cycle) methodology. This iterative approach is chosen due to the evolving nature of the Zambian transport sector and the need for continuous stakeholder feedback.

The development is organized into two-week sprints, each focusing on a specific set of deliverables:

1. Sprint 1: Core Portal & Database: Developing the operator registration and route management architecture.
2. Sprint 2: Booking Engine: Implementing the seat selection logic and real-time inventory synchronization.
3. Sprint 3: Payment Integration: Connecting the system to Zambian mobile money APIs (Airtel/MTN).
4. Sprint 4: UI/UX & Notifications: Refining the traveler interface and automated email/SMS receipt generation.

This methodology is backed by an Object-Oriented Analysis and Design (OOAD) approach. By modeling the system using objects such as "Bus," "Route," "Ticket," and "User," the system ensures modularity and scalability. These objects interact through well-defined procedures, ensuring that data moves securely from the traveler's input to the operator's management dashboard.

3.2.1. Justification for Agile Scrum

The choice of Agile Scrum is directly informed by the literature review findings. As noted in the review of Shimomba et al. (2025), a major barrier to adoption in Zambia is "low awareness" and the need for user education. An Agile approach allows for continuous feedback from pilot users (students and operators) during bi-weekly sprints, ensuring the interface (UI) evolves to match the low-digital-literacy requirements highlighted by Ngoma et al. (2019).

Furthermore, the review of AfricanBus and GIG Mobility revealed the necessity of integrating with multiple payment APIs (Airtel/MTN). Agile's iterative nature allows the team to integrate and test these payment gateways incrementally (Sprint 3) without delaying the core booking engine (Sprint 2). This reduces the technical risk associated with third-party API integration.

3.3. Information Gathering and Analysis

To ensure the system meets the actual needs of the Zambian market, a three-step data collection process was implemented.

3.3.1. Technical Benchmarking of Existing Systems

The primary phase of information gathering focused on a "System Analysis" of established platforms reviewed in the literature such as GIG Mobility, KaCyber, African bus and RedBus. By reverse-engineering the user workflows of these systems, the following core functional requirements were identified:

- Search Parameters: Ability to filter by date, price, and bus class.
- Visual Seat Selection: A grid-based UI for real-time seat inventory
- Dynamic Ticketing: Generation of unique QR codes or Reference IDs for verification.
- Operator Dashboards: Tools for bus companies to update fares instantly.

3.3.2. Field Research: Interviews and Surveys

Following the technical analysis, field research will be conducted to validate these features in the local context. Techniques such as user surveys will be employed and distributed to travelers (Students) to determine their digital readiness, preferred payment methods and the specific information they find missing when visiting a physical station, information such as departure time and bus quality.

3.3.3. Requirements Analysis and Validation

The data gathered from existing systems and field stakeholders is analyzed to form a Requirements Specification. This involves categorizing needs into functional requirements (what the system must do) and non-functional requirements (how the system should perform, such as security and speed). These requirements are then validated against the project objectives to ensure the solution addresses the problem of fare opacity and accessibility in Zambia's inter-province transport.

3.3.4. The data collected from technical benchmarking and stakeholder surveys will be analyzed using thematic analysis. For qualitative data (interview notes regarding user frustrations), recurring themes (e.g., "concern about double booking") will be coded

and grouped. Quantitative data (e.g., preferred payment method statistics) will be analyzed using descriptive statistics to prioritize non-functional requirements. This dual-analysis ensures the final system design is grounded in empirical evidence from the target user base.

3.4. Requirements Specification

This phase defines the characteristics of the BookMyBus Zambia Management System categorized into high-level user requirements and detailed system requirements.

3.4.1. User Requirements

The high-level expectations from the different actors are as follows:

- Guest (Unauthorized User): Can search for bus routes, view departure schedules, compare fare prices between different operators and view bus amenities if on offer.
- Traveler (Authorized User): Access to all guest features in addition to the ability to select specific seats, make secure payments via mobile money and receive digital tickets via email.
- Bus Operators: Ability to manage bus fleets, update dynamic fare structures and view real-time passenger manifests.
- System Administration: Verify and onboard new operators, monitor system performance and ensure data integrity across the platform.

3.4.2. Functional Requirements

1. Search & Comparison Engine: The system shall allow users to search for routes based on origin, destination, and date.
2. Authentication Logic: The system shall restrict the "Seat Selection" and "Payment" modules to logged-in users only.
3. Booking & Reservation: The system shall allow authenticated users to select available seats from a visual grid and hold them for a maximum of 10 minutes pending payment.
4. Payment Gateway Integration: The system shall process transactions through Zambian mobile money APIs (MTN/Airtel) and generate a unique Reference ID upon success.
5. Operator Management Portal: The system shall provide a dashboard for operators to publish bus schedules and adjust fares based on demand.

3.4.3. Non-Functional Requirements

- Security: The system must use SSL encryption for all data transfers and comply with the Zambia Data Protection Act regarding user personal information.
- Performance: Search results for bus routes must be returned within 5 seconds under normal network conditions.
- Availability: The system shall maintain an uptime of 98%, ensuring travelers can book tickets 24/7.
- Usability: The interface must be mobile-responsive to accommodate the high volume of smartphone users in Zambia.

3.4.4. Software and Hardware Requirements

i. Hardware Requirements

- Development Machine: Intel Core i5, 8GB RAM, 500GB SSD.
- Hosting Server: Cloud-based instance such as AWS/Firebase with auto-scaling.
- Testing Devices: Android and iOS smartphones for mobile money testing.

ii. Software requirements

- Frontend/Backend: React.js, PHP (Laravel Framework).
- Database: PostgreSQL.
- Development Tools: VS Code, Postman, Git.

3.5. System Analysis

System analysis uses graphical tools to model the requirements elicited in the previous sections. Following the OOAD approach, the following models represent the system logic:

3.5.1. Use Case Model

The system architecture differentiates between the “Guest” and “Registered Traveler.” The “Book Ticket” use case includes the “Login” use case as a mandatory dependency.

Use Case	Search and Compare Routes
Actor	Guest, Traveler
Description	Allows searching for bus routes and comparing fares from different operator.
Stimulus	User enters origin, destination, and date into the search form.
Response	System displays a sortable table of available buses and prices.

Use Case	Book Ticket
Actor	Traveler (Authenticated)
Description	Allows seat selection, payment, and receipt of a digital ticket.
Stimulus	User selects a seat and clicks “Proceed to Payment.”
Response	System processes payment via mobile money and sends a digital ticket.

3.5.2. Entity Relationship Modeling

To handle the information effectively, the following entities and relationships are handled:

- User Entity: Stores credentials and profile (Guest vs Registered).
- Operator Entity: Stores company details and verification status.
- Bus Entity: Linked to an Operator; stores seat capacity and amenities.

- Route Entity: Defines the path between two Zambian towns.
- Booking Entity: Links a User, a specific Seat, and a Payment record.

3.5.3. Data Flow Analysis

The system follows a 3-tiers flow:

1. Input: User provides route details or login credentials.
2. Process: The system validates credentials or queries the database for active bus schedules.
3. Output: For Guests, the output is a fare comparison table. For Travelers, the output is a validated digital ticket.

3.6. Conclusion

This chapter has detailed the research methodology and system requirements for the BookMyBus Zambia Management System. By adopting the Agile Scrum methodology and a structured OOAD approach, the project ensures that the core problem, lack of transparency in bus ticketing, is solved through a secure, authenticated platform. The analysis of existing systems combined with planned stakeholder surveys has established a robust framework for the system's functionality. The project further ensures that the solution is grounded in literature and local user needs. The functional and non-functional requirements defined here, along with the OOAD models, provide the blueprint for the subsequent **(System Design)**, where these logical models will be transformed into concrete architectural and interface designs.

Chapter 4: system design

4.1 Introduction

This chapter presents the system design of the BookMyBus Zambia Management System. The design phase translates the requirements identified in Chapter 3 into a structured architecture that can be implemented using modern software technologies.

The system follows an Object-Oriented Analysis and Design (OOAD) approach where the system is structured around objects such as User, Bus, Route, Booking, and Payment. These components interact to support the core system operations such as searching routes, reserving seats, and processing payments.

This chapter presents the architecture, system modules, database structure, and design models including context diagrams, use case diagrams, sequence diagrams, activity diagrams and data flow diagrams.

4.2 Analysis of the System

Analysis focuses on what the system should do, while design focuses on how the system will achieve it. The system must:

Functional Requirements

- Provide passengers with a centralized platform to search routes, view fares, and book tickets.
- Enable secure, flexible payment options (mobile money, Zanaco payments).
- Support offline-first functionality to accommodate low-connectivity regions.
- Offer operator dashboards for schedule management, fare setting, and reporting.
- Ensure compliance with data protection and payment security standards (e.g., PCI-DSS).
- Deliver multilingual and localized user interfaces to improve adoption.

Non-Functional Requirements

- **Reliability:** Must function even in unstable network conditions.
- **Scalability:** Should support growth across multiple operators and regions.

- **Security:** Protect user data and transactions from fraud or breaches.
- **Usability:** Simple, mobile-first design for users with varying digital literacy.
- **Interoperability:** Standardized APIs for integration with other transport or payment systems.

Stakeholders

- **Passengers:** Need transparency, convenience, and trust.
- **Bus Operators:** Require tools for managing operations and building credibility.
- **Regulators/Polymakers:** Demand compliance, reporting, and oversight.

System Administrators: Oversee governance, security, and system performance.

The existing bus ticketing system in many parts of Zambia is still manual. Travelers must visit physical bus stations to inquire about routes, prices, and availability. This process often results in long queues, lack of transparency in ticket pricing, and inefficient seat allocation.

The proposed BookMyBus Zambia Management System provides a centralized digital platform that allows travelers to search routes, compare fares, reserve seats, and pay through mobile money services such as MTN and Airtel.

The system must support the following major functions:

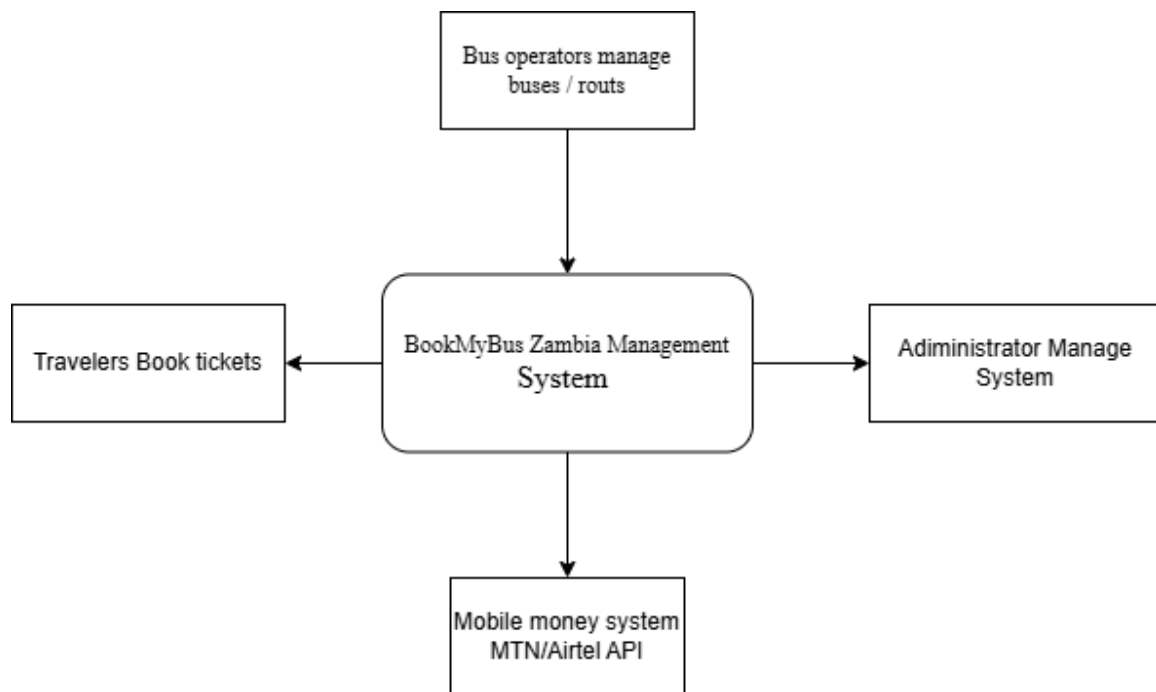
- Searching bus routes
- Comparing fares between operators
- Selecting seats
- Processing ticket payments
- Managing bus routes and schedules
- Generating digital tickets

4.3 Context Model of the System

The context model illustrates the interaction between the system and external entities.

The primary external entities interacting with the system include travelers, bus operators, system administrators, and mobile money payment services.

Context Diagram



4.4 Design methods

4.4.1 Architectural Design

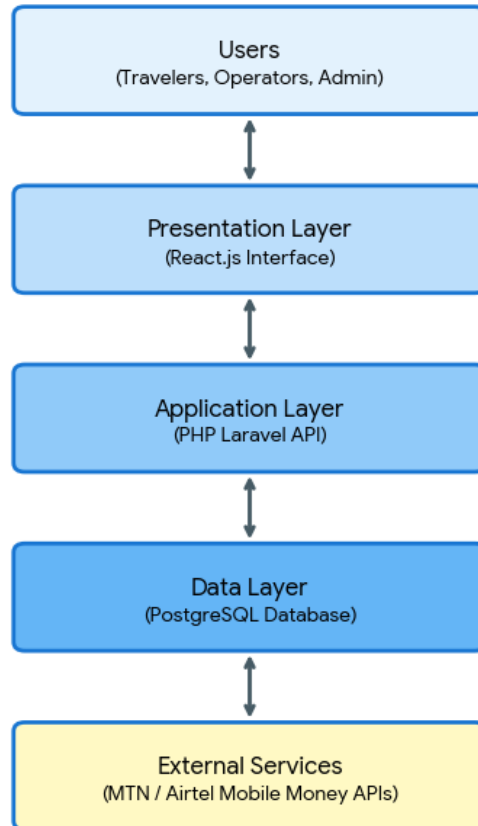
The BookMyBus Zambia Management System uses a three-tier client-server architecture.

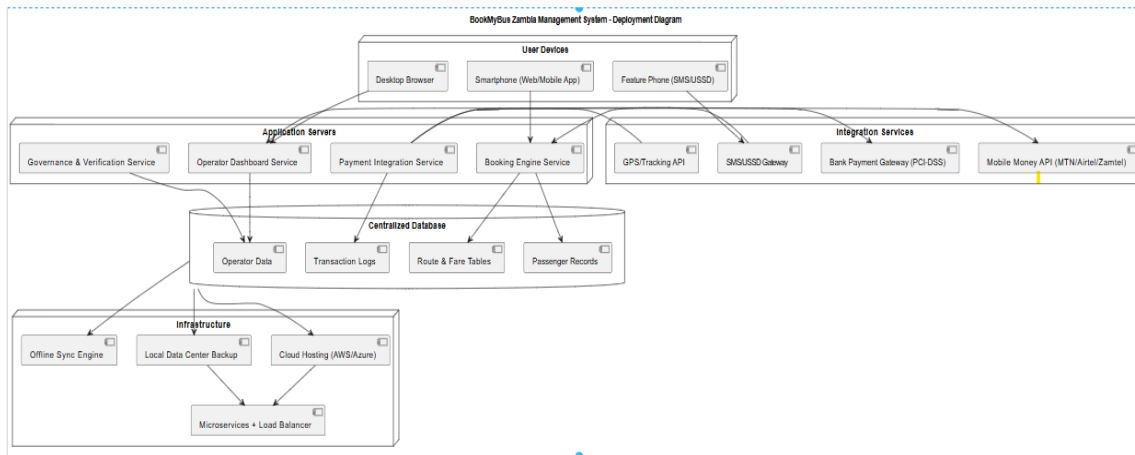
1. Presentation Layer – Provides the user interface implemented using React.js.
2. Application Layer – Handles system logic such as booking, authentication and payment processing using Laravel.
3. Data Layer –: PostgreSQL database ensuring relational integrity for fares, bookings, and payments. Additional services include Firebase for hosting and authentication, and Zambian payment gateway APIs for transactions.

This architecture was chosen for scalability, modularity, and compatibility with Zambia's connectivity conditions.

Architecture Diagram

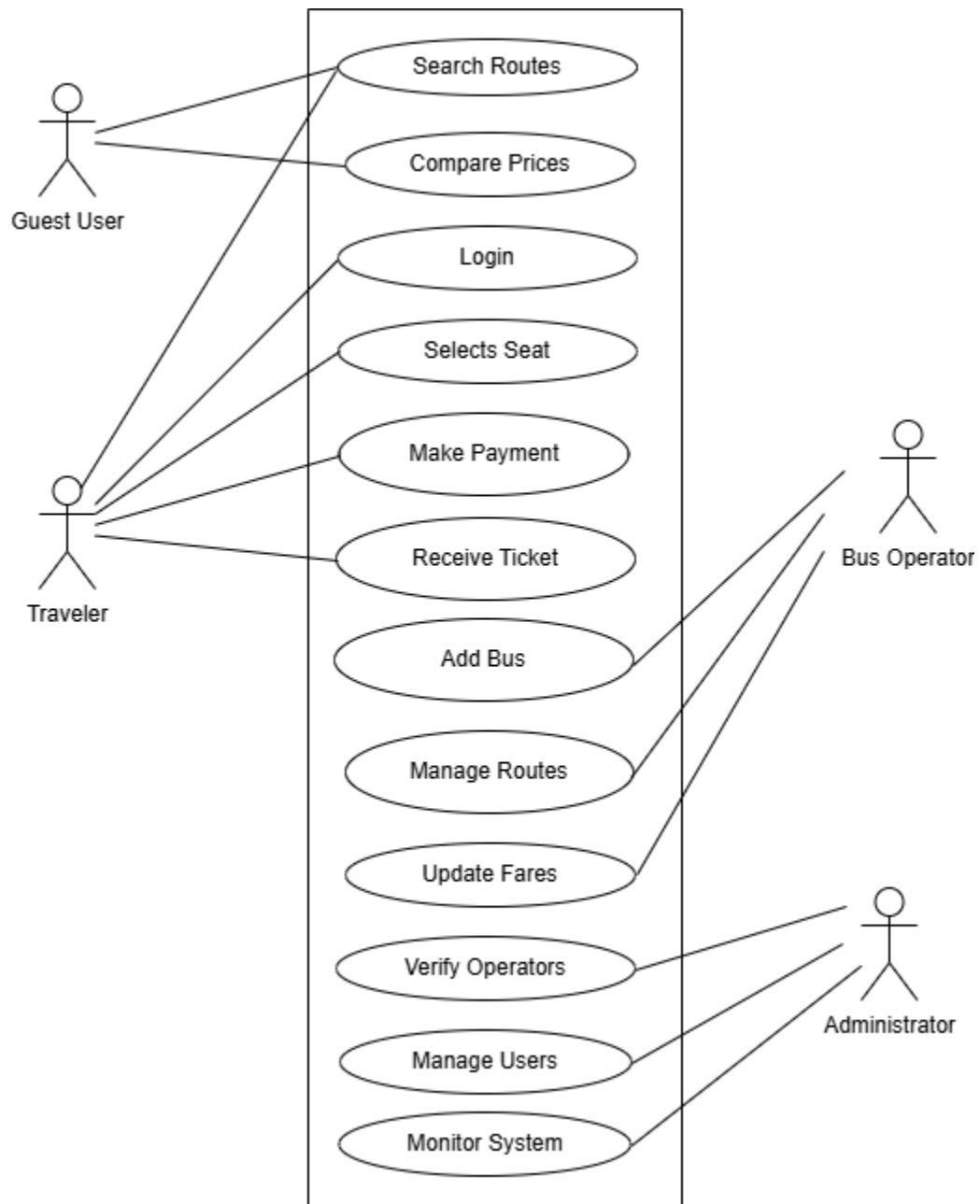
BookMyBus Zambia: 4-Tier Architecture





Use Case Diagram

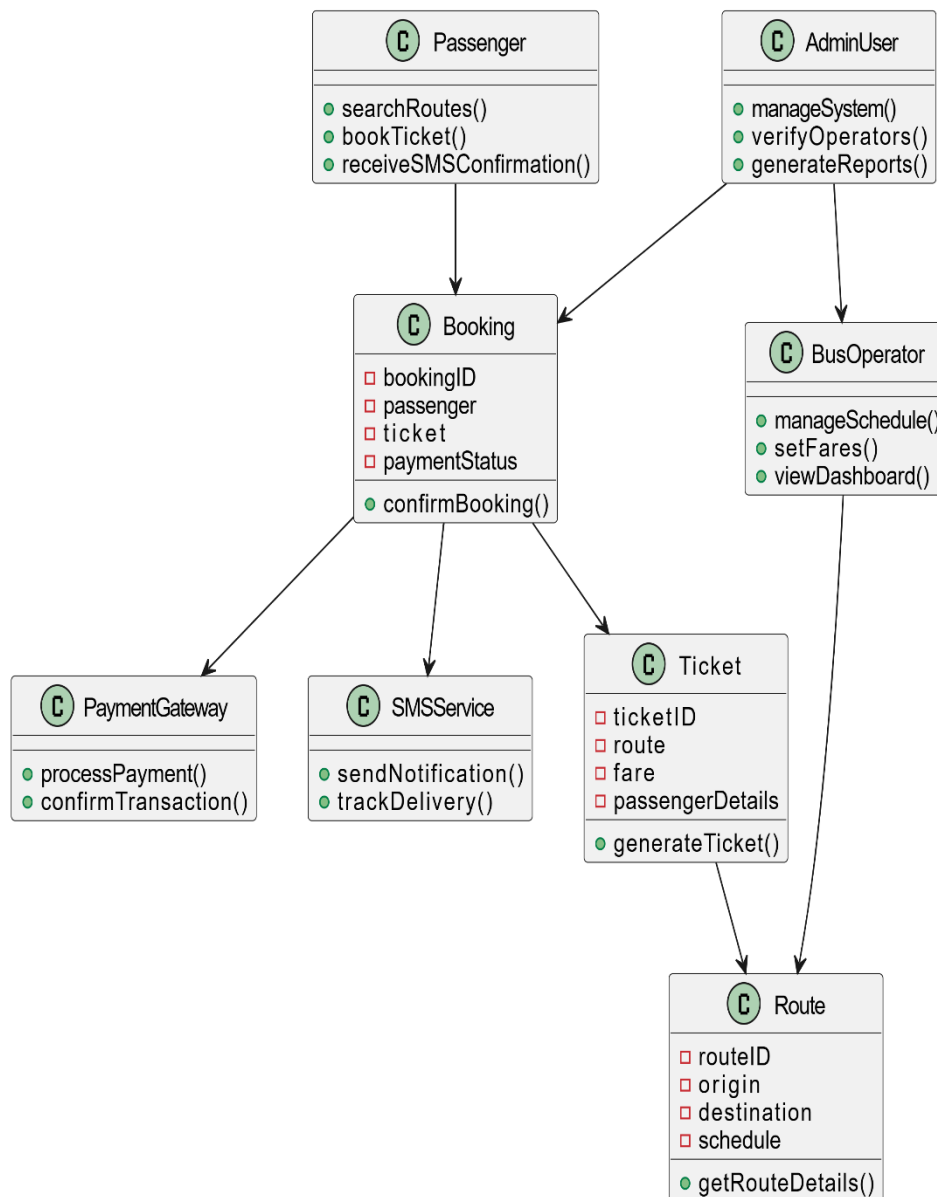
- Traveler: Search routes, compare fares, book seats, pay online.
- Operator: Register, publish fares, manage seat inventory.
- Administrator: Verify operators, monitor system, generate reports.



Class Diagram:

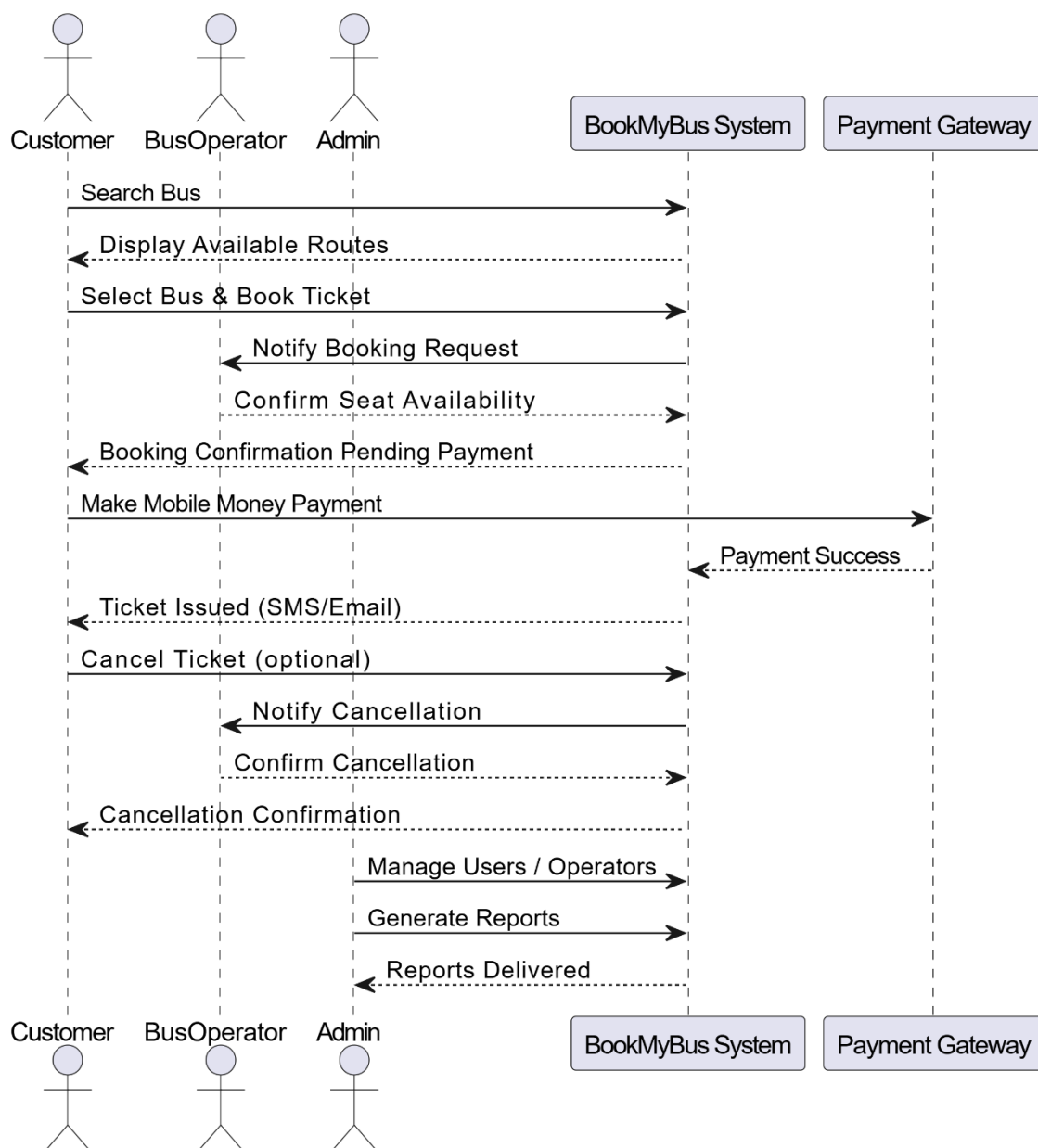
- Core classes: User, Operator, Route, Booking, Payment, Ticket.

BookMyBus Zambia - Class Diagram

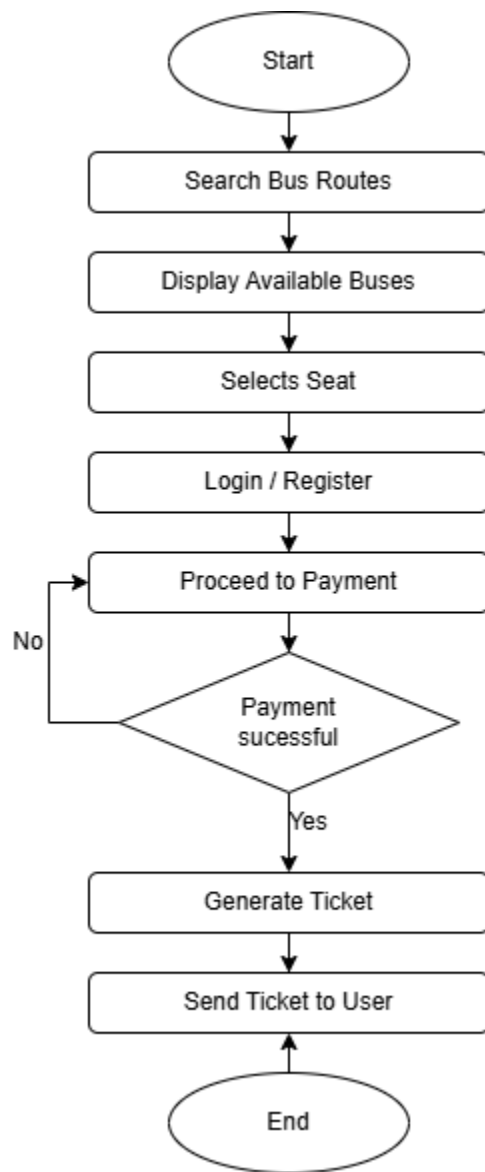


Sequence Diagram – Booking Process

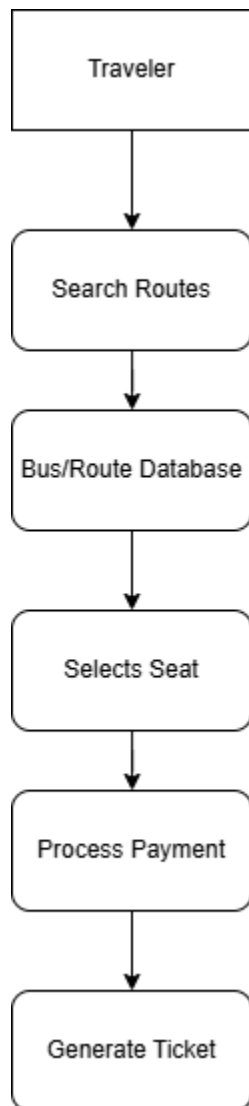
1. Traveler searches route.
2. System retrieves fares and seat availability.
3. Traveler selects seat and initiates payment.
4. Payment gateway validates transaction.
5. System confirms booking and issues digital ticket.



Activity Diagram – Booking Process

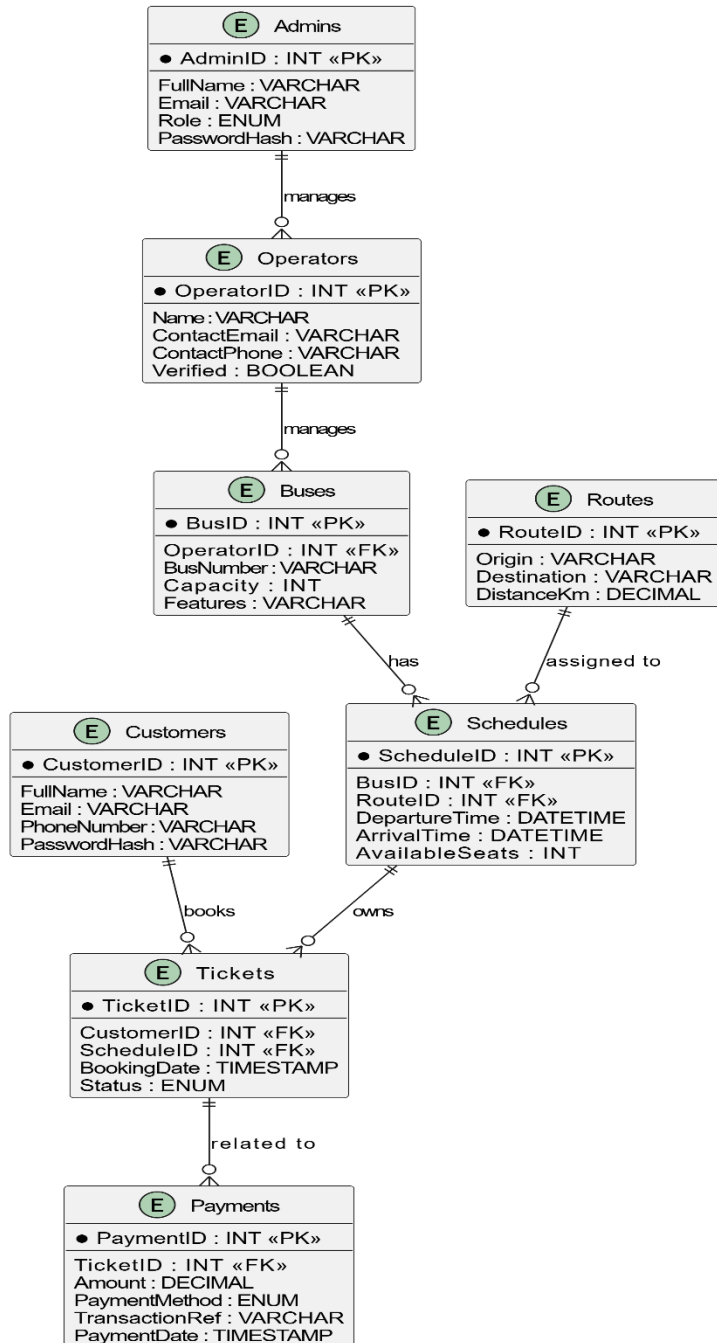


Data Flow Diagram



Database Schema:

- Tables: Users, Operators, Routes, Bookings, Payments, Tickets.
- Relationships: One operator → many routes; one booking → one payment; one traveller → many bookings.



4.4.3 Physical Design

Physical design describes how the system will operate in practice in terms of **data input, processing, storage, and output**. It focuses on how users interact with the system interface and how data flows through the system components.

The physical design of the **BookMyBus Zambia Management System** focuses on three major areas:

- Input Design
- Output Design
- Data Design and Storage

Input Design

Input design describes how users enter data into the system. The BookMyBus system provides several user-friendly forms that allow users to interact with the platform efficiently.

The main input interfaces include:

1. User Registration Form

This form allows travelers to create accounts by entering details such as:

- Full Name
- Email Address
- Phone Number
- Password

The system validates the entered information before storing it in the database.

2. Login Form

Registered users access the system by entering their:

- Username or Email
- Password

The system verifies the credentials before granting access to the booking platform.

3. Route Search Form

Travelers search for bus routes by providing:

- Origin location

- Destination location
- Travel date

The system retrieves matching routes from the database and displays them to the user.

4. Seat Selection Interface

After selecting a bus route, users can view a **visual seat layout** of the bus showing available and reserved seats. The traveler can select an available seat before proceeding to payment.

Seat Layout Example

Driver

```
[1] [2] [3] [4]
[5] [6] [7] [8]
[9] [10] [11] [12]
[13] [14] [15] [16]
```

Legend:

Available Seat = Green

Booked Seat = Red

Selected Seat = Blue

Output Design

Output design describes how information is presented to the users.

The BookMyBus system generates several outputs including:

1. Bus Search Results

After searching for routes, the system displays:

- Bus operator name
- Departure time
- Ticket price
- Available seats

Example output format:

Operator	Route	Departure Time	Price
Power Tools Bus	kitwe → Ndola	08:00	K250
Rayon Motors	Lusaka → Kitwe	09:30	K300

2. Booking Confirmation

After successful payment, the system generates a booking confirmation message showing:

- Booking ID
- Seat Number
- Route
- Travel Date
- Payment Reference Number

3. Digital Ticket

A digital ticket is generated and sent to the traveler via:

- Email
- SMS notification

The ticket includes a **unique QR code or reference number** that can be used for verification at the bus station.

Example ticket structure:

BOOKMYBUS ZAMBIA TICKET

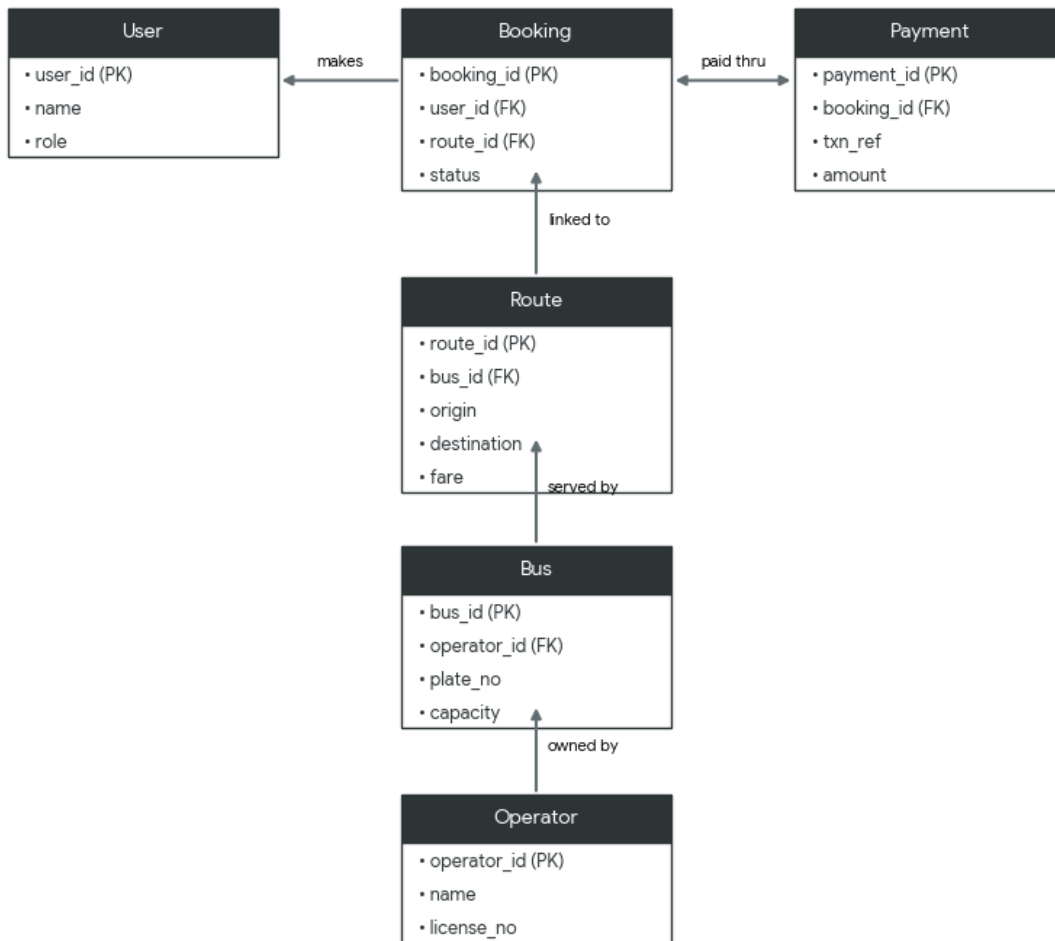
Passenger Name: John Katanga
Route: Lusaka → Kitwe
Bus Operator: power tools Motors
Seat Number: 12
Departure Time: 09:30
Reference ID: BMZ123456

Data Design and Storage

The system stores information in a **PostgreSQL relational database**. The database is designed to store and manage different types of data used by the system.

The main database tables include:

Table Name	Description
Users	Stores traveler accounts
Operators	Stores bus company information
Buses	Stores bus details and capacity
Routes	Stores travel routes
Bookings	Stores ticket reservations
Payments	Stores payment transactions



Data Security

To protect user data and financial transactions, the system implements several security measures:

- **Security:**

- PCI-DSS compliance for payment handling.
- End-to-end encryption for data in transit.
- Secure authentication (multi-factor for operators/admins).
- Audit logs for regulatory compliance.

- **Validation:**

- Input validation (dates, seat numbers, payment amounts).
- Operator verification (only registered operators can publish schedules).
- Payment validation (transaction success/failure codes).

- **Transformation:**

- Raw payment data → standardized transaction record.
- Route/fare data → user-friendly display (localized language, currency).

Analytics → dashboards for operators and regulators.

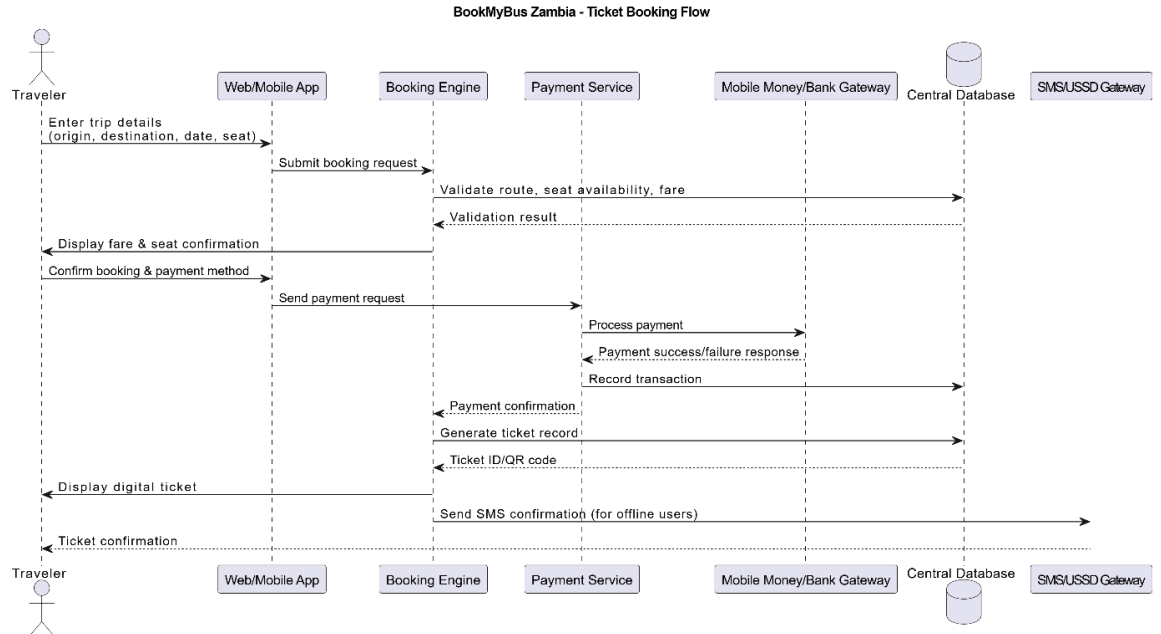


Table	Key Indexes	Storage	Constraints and Notes
Customers	PK: CustomerID Unique: Email, Phone Number	InnoDB	Passwords stored as hashes; language for localization
Operators	PK: OperatorsID Index: Verified	InnoDB	Used for governance and dashboard access
Buses	PK: BusID FK: OperatorID	InnoDB	Indexed for fast Schedule lookup
Routes	PK RouteID Index: Origin, Destination	InnoDB	Time-based indexing for availability
Tickets	PK: TicketsID FKs: CustomerID, ScheduleID Index: Status	InnoDB	ENUM for lifecycle states
Payments	PK: PaymentId FK: TicketID Index: TransactionRef, Status	InnoDB	PCI-Dsscompliance mobile money support
Admins	PK: AdmiID Unique: Email	InnoDB	Role-Based access control

These mechanisms ensure that the system maintains confidentiality, integrity, and availability of user data.

4.6 System Maintenance Considerations

- **Operator Updates:** Admin portal for fare adjustments and operator verification.
- **Scalability:** Designed to expand beyond 20 routes and 5 operators.
- **Error Logging:** Implement centralized logging for quick detection of failure in booking or payments flows
- **Database Optimization:** Use indexing, partitioning and query optimization to keep searches fast
- **Payment Gateway Compliance:** Maintain PCI-DSS compliance with periodic audits and updates.
- **Security:** PCI-DSS compliant payment integration, audit logging, and role-based access control.

4.5 Conclusion

This chapter presented the system design of the BookMyBus Zambia Management System.

The design described the architecture, system modules, database structures required to implement the platform and directly addressing inefficiencies in manual fare inquiry and ticketing.

The system uses a three-tier architecture combined with object-oriented design principles to ensure scalability, security, and maintainability. The models and diagrams presented in this chapter provide the blueprint for implementing the system in the next stage of the project.

The review emphasizes that while digital bus ticketing platforms promise efficiency and transparency, many fail in practice because of contextual challenges such as low connectivity, limited digital literacy, and weak trust frameworks. The **BookMyBus Zambia Management System** is presented as a solution that directly addresses these shortcomings through **localized, secure, and resilient design choices**. Specifically, it highlights benefits for **travellers (ease of booking, transparency)**, **operators (governance tools, payment integration)**, and **policymakers (standardized data and compliance)**.